Horse Racing ML Pipeline — Project Documentation

Avery Watts — 2026

1. Project Overview

This project builds a logistic regression model to predict horse race outcomes using historical performance data. The pipeline involves scraping horse profile PDFs from Equibase, extracting structured statistics using an LLM agent (Google Gemini), engineering predictors to minimize multicollinearity, and merging the data with race-level CSV data for modeling. A secondary pipeline handles jockey data with the same architecture.

2. Files Used

Input Files

- **horse-info/ (folder)** — 2868 PDFs downloaded from Equibase, one profile PDF and one statistics PDF per horse. Named as horsename.pdf and horsenamehorsename.pdf.
- **horse_names_used.csv** — Master list of 1463 unique horse names used in the race dataset. Single column: horse_name2. No parenthesis suffixes (e.g. PR, KY, IRE).
- **parenthesis_codes.csv** — List of ~30 possible track/origin suffixes that Equibase appends to horse names in parentheses (e.g. PR, KY, IRE, FR, GB, ARG). Used to brute-force match horse names between datasets.
- **output_clean205.csv** — Race-level dataset. Each row is one horse in one race. Contains: date, race_event_name, race_number, horse_name, jockey, last_raced, weight, start_position, finish_position, odds, race_date, last_raced_date, days_since_last_race.
- **h_a_prompt.txt** — System prompt for the Gemini extraction agent. Instructs the model on exactly what to extract from the PDFs and how to return it as a JSON object.

Output / Intermediate Files

- **ha-5.xlsx / ha-8.csv** — Output of the Gemini extraction agent. One row per horse. Contains: horse_name (with parenthesis suffix), adjusted_starts, adjusted_firsts, adjusted_seconds, adjusted_thirds, speed_figure_sum, yearly_earnings_2018 through 2025, win_rate, Place_rate, 3yr_earnings, career_earnings, earnings_per_start.

- **horse_map.csv** — Dictionary mapping clean horse names (no suffix) to their full Equibase names (with suffix). Built by the name-matching pipeline. Reusable for future merges.
- **unmatched_pdfs.txt** — List of PDF filenames that could not be fuzzy-matched to any horse name in horse_names_used.csv. Used to manually identify and rename problem files.
- **output_clean205_merged-4.csv** — Final merged dataset. Race-level rows with horse profile statistics joined in. Ready for feature engineering and modeling.

3. Gemini Extraction Agent

3.1 What It Does

Each horse on Equibase has two PDFs: a results/race history PDF and a statistics breakdown PDF. The agent sends both PDFs in a single API call to Gemini (gemini-3-flash-preview) and receives a structured JSON object back with the extracted statistics. This avoids manual data entry for 1400+ horses.

3.2 The Prompt (h_a_prompt.txt)

The system prompt instructs the model to:

- Extract career starts, firsts, seconds, thirds from the CAREER STATISTICS section.
- Apply a March 2026 cutoff: subtract any race dated after 02/28/2026 from the career totals, and adjust the place counts based on the Finish column (1 = subtract from firsts, 2 = seconds, 3 = thirds, 4+ = no change).
- Calculate speed_figure_sum: ignore post-cutoff races, sum the E (speed figure) column of the most recent 4 valid races. If fewer than 4 exist, sum all available.
- Extract yearly earnings from 2018-2025 from the statistics breakdown table. Return 0 for missing years.
- Return only a JSON object with exact field names — no markdown, no explanation, no code blocks.

3.3 Why the Cutoff Was Needed

The race dataset covers races up to 02/28/2026. Equibase career statistics include all races to date, which means March 2026 races are included in the career totals but not in the race dataset. Without the cutoff adjustment, the career stats would be slightly ahead of the race data in time, creating data leakage. The agent handles this correction automatically by scanning the results table for post-cutoff races and subtracting them.

3.4 Async Architecture

Processing 1414 horses sequentially would take ~3.8 hours at ~9.5 seconds per horse. The agent uses Python asyncio to run up to 50 concurrent API calls simultaneously, reducing total runtime to approximately 25-30 minutes. Key components:

- **asyncio.Semaphore(50)** — Limits concurrent calls to 50 to avoid hitting the Gemini rate limit (RPM cap).
- **asyncio.gather()** — Fires all tasks concurrently and waits for all to complete.
- **asyncio.to_thread()** — Runs the synchronous Gemini SDK call in a thread pool so it doesn't block the event loop.
- **Retry logic** — On 429 (rate limit) errors, backs off 30s then 60s before failing. Prevents silent data loss.
- **Progress saving** — Saves to Excel every 10 completions using a save_lock (asyncio.Lock) to prevent concurrent writes corrupting the file.

4. PDF-to-Horse Name Matching

4.1 The Problem

The 2868 PDFs need to be grouped into pairs (one profile PDF + one stats PDF per horse) and matched to the correct horse name. PDF filenames were inconsistent: some were clean (eclesiastes.pdf), some had spaces (aspirational aspirational.pdf instead of aspirationalaspirational.pdf), some had random strings (21we12234.pdf), and the double-name convention was not perfectly uniform.

4.2 The Solution: Fuzzy Matching to horse_names_used.csv

Rather than relying on filename sorting or pairing conventions, the agent matches each PDF filename (stem, lowercased) against the master horse name list using rapidfuzz partial_ratio scoring. Partial ratio was chosen over WRatio because it correctly handles the case where a horse name is a substring of the filename (e.g. 'ruse' inside 'ruseruse'). A threshold of 45 was used — loose enough to catch all valid matches, while truly invalid filenames (e.g. 21we12234) score below it and are saved to unmatched_pdfs.txt.

4.3 Why Not Sort by Filename?

Alphabetical sorting was attempted first. It failed because some double-name files sort before the single-name file (e.g. 'aspirational aspirational.pdf' sorts before 'aspirational.pdf' due to the space character). This caused cascading mispairings from that point onward in the sorted list. A length-based sort was tried next but also failed because unrelated short-named horses sorted together. Fuzzy matching against the known name list was the only robust solution.

4.3 Issue: ~63 Horses Initially Unmatched

After the first full run of the fuzzy matching agent, approximately 63 horses failed to match any PDF in the horse-info folder. These were saved to unmatched_pdfs.txt. The root cause was inconsistent PDF filenames that scored too low against the horse name list — either due to unusual characters, abbreviations, or naming conventions that differed from the master CSV.

4.4 Fix: Rename PDFs and Lower Threshold

Rather than entering 63 horses by hand, the PDFs were renamed to more closely match the horse names in the CSV and placed in a separate folder (redone names/). A second agent run was done against this folder using a separate name list (unused_horses.csv) containing only the still-unmatched horses. The fuzzy threshold was lowered from 45 to 35 to accommodate the remaining edge cases. The agent code was otherwise identical to the main run. Output was saved to ha-7.xlsx and later merged into the main dataset.

4.5 Post-Merge Residual Issues (7 Horses)

After merging the horse information into the race dataset, 7 horses still had no data attached:

- **3 horses** had their information present in the horse info dataset but failed to match during the merge step due to subtle name discrepancies. These were fixed by manually correcting the name in the map.
- **4 horses** had simply never had their Equibase PDFs downloaded. Rather than re-running the full agent pipeline for just 4 horses, the statistics were entered manually into the dataset. This was the pragmatic choice — running the agent for 4 horses would have required the same setup overhead as a full batch run, while manual entry took minutes.

5. Horse Name Mapping Dictionary

5.1 The Problem

The race CSV uses plain horse names (e.g. Eclesiastes). The Equibase ha-5 dataset appends origin/track suffixes in parentheses (e.g. Eclesiastes (PR)). There are approximately 30 different possible suffixes and it was unknown which horses had them, which suffix applied to which horse, or which horses had no suffix at all. A simple string strip on both sides failed because the race CSV also sometimes contained parenthetical suffixes with different meanings.

5.2 The Dictionary Approach

A horse_map dictionary was built using a two-pass approach:

- **Pass 1 — Exact match:** Iterate through horse_names_used.csv. If the name appears as-is in ha-5, map it directly (horse -> horse). These are horses with no suffix.
- **Pass 2 — Brute force parenthesis combos:** For remaining unmatched horses, try every combination of horse_name + ' (' + suffix + ')' against the ha-5 horse_name column. ~30 codes x ~500 unmatched horses = ~15,000 combinations, all checked instantly in Python. When a match is found, store horse -> horse (suffix) in the map.

5.3 Why This Was the Right Approach

The key insight is that the map is built once using ground truth exact matches — not guesses or fuzzy similarity scores. This means every entry in the map is definitively correct. The brute force is not expensive computationally and only runs at build time. The resulting horse_map.csv is a reusable artifact: for any future race dataset, you simply load the map and join on it without rerunning the matching logic. Fuzzy matching was deliberately avoided in this step because similar horse names (e.g. Fear vs Fearnought, Attack vs Attacked) would produce false matches that corrupt the model data.

6. Feature Engineering & Multicollinearity

6.1 The Multicollinearity Problem

In logistic regression, multicollinearity occurs when two or more predictors are highly correlated. This inflates standard errors, makes coefficients unstable, and can cause the optimizer to produce nonsensical weights (e.g. one predictor going positive while a correlated one goes negative). The raw data contains obvious collinear pairs: adjusted_firsts correlates with adjusted_starts (more starts = more chances to win), and career_earnings correlates with win_rate (winning earns money).

6.2 Engineered Predictors

Raw counts were replaced with rates and aggregates that measure genuinely different things:

- **win_rate** = adjusted_firsts / adjusted_starts. Measures quality of wins independent of volume.
- **place_rate** = (firsts + seconds + thirds) / starts. Measures overall consistency, not just wins.

- **speed_figure_sum** = sum of last 4 E column values (pre-cutoff). Measures raw recent pace performance — orthogonal to win rate.
- **3yr_earnings** = sum of 2023 + 2024 + 2025 earnings. Captures recent class/form in dollar terms.
- **career_earnings** = total earnings across all years. Captures long-term class.
- **earnings_per_start** = career_earnings / adjusted_starts. Normalizes earnings by race volume.
- **adjusted_starts** — Kept as a standalone predictor for experience/volume signal.

6.3 What Was Dropped

- Raw adjusted_firsts, adjusted_seconds, adjusted_thirds — replaced by rates.
- Individual yearly earnings columns (2018-2025) — replaced by 3yr_earnings and career_earnings aggregates.
- yearly_earnings_2026 — too sparse and incomplete.

7. Final Merge Pipeline

The final merge joins race-level data with horse profile statistics using the horse_map dictionary:

- Load horse_names_used.csv, output_clean205.csv, ha-8.csv, parenthesis_codes.csv.
- Build horse_map via Pass 1 (exact) and Pass 2 (brute force parenthesis combos).
- For each unique horse in the race CSV, look it up in horse_map to get the ha-8 key, then copy the relevant columns into the race row.
- Output: output_clean205_merged-4.csv — race rows with all horse predictors attached.

8. Next Steps

- Build equivalent jockey data pipeline (same agent architecture, different predictors).
- Merge jockey stats into the race dataset using the same map approach.
- Run correlation matrix on all predictors to verify multicollinearity is within acceptable bounds ($r < 0.8$).
- Train logistic regression model on merged dataset.
- Evaluate model performance and iterate on predictor selection.